

2.2 LÖSUNG Typumwandlung

Yusuf hat die Methode `wasPassiertHier()` entworfen. Er ruft die Methode auf und erhält die unter der Methode stehende Bildschirmausgabe.

Erkläre das Verhalten der Methode.

```
void wasPassiertHier() {  
    System.out.println(3 + 5);  
    System.out.println("3" + "5")  
}
```

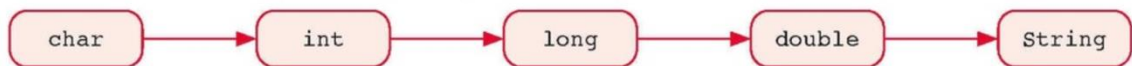
8
35

Aufgabe 1)

- c) Öffne die Online-IDE und übertrage die zwei Zeilen aus dem Methodenrumpf in ein neues Projekt (keine Klasse o.Ä. nötig). Führe das Programm aus.
- d) Ergänze zusätzlich die Zeile `System.out.println("3" + 5)`; was erwartest du als Ausgabe? Führe das Programm erneut aus und erkläre die Ausgabe.

Implizite Typumwandlung

Bei **1b)** findet eine so genannte implizite Typumwandlung statt, da die Zahl 5, welche üblicherweise vom Datentyp `int` ist, als String/Text interpretiert wird und somit das Plus-Zeichen nicht als Rechenoperator, sondern als Verkettung (Aneinanderhängen) der Zeichen genutzt wird. Die implizite Typumwandlung ist in Java nur möglich, wenn dadurch keine Informationen verloren gehen. Daher gilt in Java folgende Kette:



Jeder Datentyp kann direkt in den weiter rechts stehenden Datentyp umgewandelt werden. Ein `int` wird also z. B. ohne Umweg in einen `String` umgewandelt.

Explizite Typumwandlung

Die explizite Typumwandlung wird explizit als Anweisung im Programmtext angegeben. Hierfür muss man den gewünschten Zieldatentyp in runden Klammern vor die umzuwandelnde Variable schreiben:

```
int i = (int) 3.7;      char c = (char) 65;      i = (int) 4294967298;
```

Typumwandlung	Startwert	Endwert
double → int	3.0	3
	3.5	3
int → char	65	'A'
	9651	'△'
long → int	300	300
	4294967298	2

Achtung: Hierbei kann es zu Informationsverlust kommen.

2.3 LÖSUNG Polymorphismus

Beschreibe das Klassendiagramm rechts und erkläre, warum die folgenden Aufrufe möglich sind.

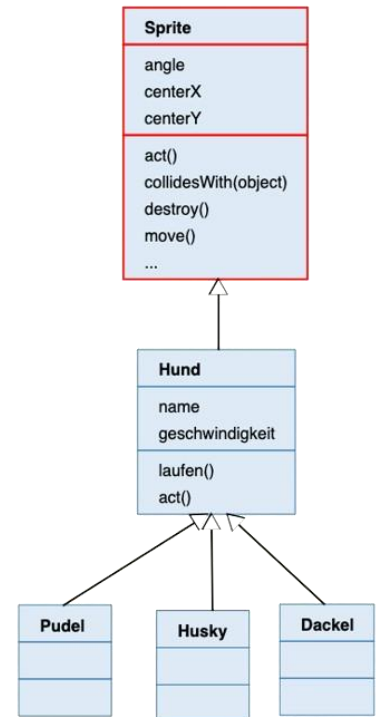
```
5 // Hunde
6 Hund hund1 = new Pudel();
7 Hund hund2 = new Dackel();
8 Hund hund3 = new Husky();
```

Aufgabe 2) 10_2-3_Hunde

- Öffne das Projekt und führe das Programm aus.
- Ergänze in der Klasse Husky das Attribut `Herrchen` und eine Methode `herrchenAusgeben()` und versuche diese im Hauptprogramm aufzurufen.
- Was fällt dir auf?

Polymorphismus bei höheren Datentypen

Speichert man ein Objekt der **Unterklasse** in einer Variablen vom Datentyp der **Oberklasse**, so ist es nicht mehr möglich, Attribute und Methoden zu nutzen, die nur auf der Unterklasse definiert sind. Stattdessen wird das Objekt der Unterklasse wie sein Behälter (mit dem Datentyp der Oberklasse) behandelt. Hier hat also eine implizite Typumwandlung stattgefunden. Diese spezielle Art der Typumwandlung, bei der ein Objekt gleichzeitig mehrere Typen hat, bezeichnet man auch als **Polymorphie**.



Aufgabe 2) 10_2-3_Hunde

- Kommentiere die Zeilen 6-21 im Hauptprogramm aus, um diese nicht mehr auszuführen. Kommentiere die Zeilen 24-30 ein, um nun diese auszuführen. Starte das Programm
- Beobachte was passiert. Untersuche die Klassen `Hund`, `Pudel`, `Husky`, `Dackel` und erkläre, wie das Verhalten zustande kommt.
- Ergänze in der Klasse `Hund` nun ebenfalls die Methode `herrchenAusgeben()` und lasse hier "Ein generischer Hund hat leider kein Herrchen" ausgeben. Rufe die Methoden auf Objekten der jeweiligen Klassen auf.

Die Unterklassen ändern den Attributwert von Geschwindigkeit, aber es wird die Methode der Oberklasse ausgeführt.

Bei `herrchenAusgeben()` wird, sofern vorhanden, die überschriebene Methode der Unterklasse ausgeführt.

Dynamische Bindung von Objekten

Wird ein Objekt der Unterklasse im Behälter der Oberklasse gespeichert, so sind nur die Methoden der **Oberklasse** verfügbar. Diese können jedoch von den Unterklassen **überschrieben** werden. Aus diesem Grund muss während der Ausführung des Programms geprüft werden, ob die Methode überschrieben ist. Ist das der Fall, so wird die auszuführende Methode der **Unterklasse** ermittelt und ausgeführt. Diese Art der Vermittlung nennt man auch **dynamische Bindung**. Um bereits im Programmcode klarzumachen, dass es sich in der Unterklasse um eine überschriebene Methode der Oberklasse handelt, nutzt man die Annotation **@Override**. Diese wird in der Zeile über der entsprechenden Methode eingefügt.